Solution: Delete Node in a BST

Problem Statement

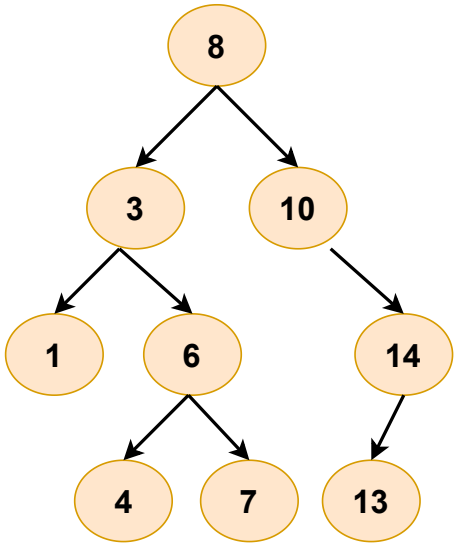
Given a `root` node of the binary search tree, and an integer value `key`, return the `root` of the updated BST after `deleting` the node having a value equal to `key`.

If BST doesn't contain the node having value equal to `key`, return the same binary tree.

Examples

Example 1:

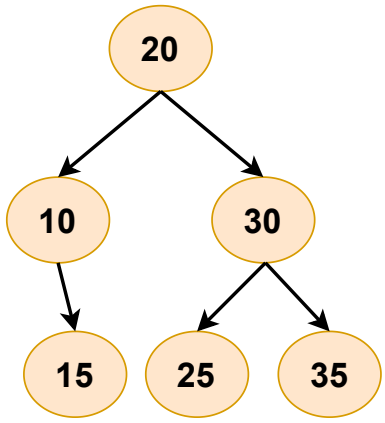
- **Input**: `[8, 3, 10, 1, 6, null, 14, null, null, 4, 7, 13]`, **key** = `3`



- **Expected Output**: `[8, 4, 10, 1, 6, null, 14, null, null, null, 7, 13]`
- **Justification**: After removing the node with value `3`, its right subtree's smallest value (`4`) replaces it to maintain the BST structure.

Example 2:

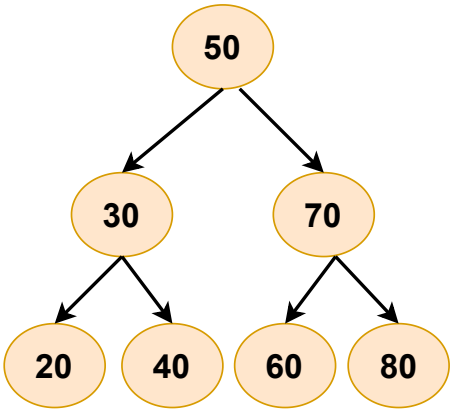
- **Input**: `[20, 10, 30, null, 15, 25, 35]`, **key** = `10`



- **Expected Output**: `[20, 15, 30, null, null, 25, 35]`
- **Justification**: The node `10` is removed and replaced by its right child `15` to keep the tree's integrity.

Example 3:

- **Input**: `[50, 30, 70, 20, 40, 60, 80]`, **key** = `10`



- **Expected Output**: `[50, 30, 70, 20, 40, 60, 80]`
- **Justification**: The BST doesn't contain the node with value 10. So, it returns the original BST.

Solution

To solve this problem, we will traverse the BST to find the target node to be deleted. If the target node has both left and right children, we'll find its in-order successor (the smallest node in its right subtree) to replace it, ensuring the BST's properties remain intact. If the target node has only one child or no children, we'll simply remove the node and connect its parent to its child if it exists.

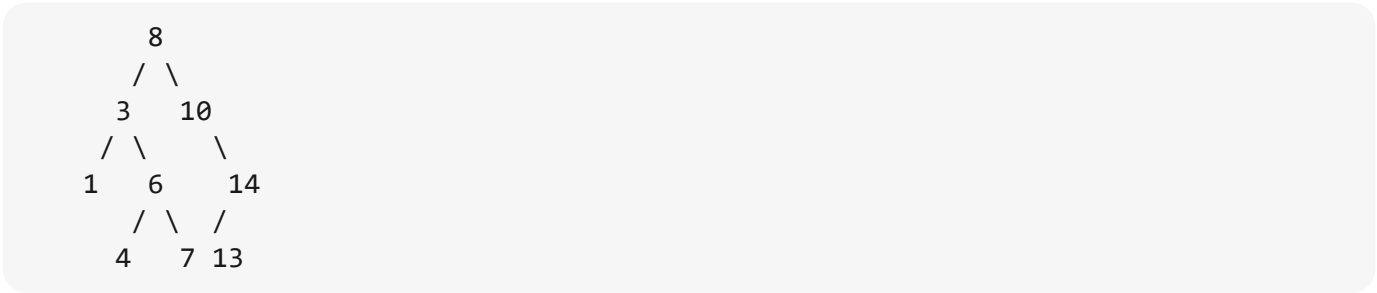
This approach works because it respects the BST properties by carefully choosing a replacement for the deleted node when necessary and properly handling leaf nodes and nodes with a single child. It's effective because it minimally disrupts the tree's structure, requiring no unnecessary movements or changes beyond those needed to preserve the BST properties.

Step-by-step Algorithm

1. **Check if the root is null**: If the root is null, there is no tree to modify, so return null.
2. **Search for the node to delete**: Traverse the tree to find the node with the value equal to the key. Keep track of the parent node as you search, as you'll need it to update the tree structure after deletion.
3. **Node not found**: If the node is not found (i.e., we reach a null pointer), return the root as is because there's nothing to delete.
4. **Handle node with two children**:
   - If the node to be deleted has both left and right children, find the in-order successor of the node, which is the smallest node in its right subtree.
   - Swap the value of the node to be deleted with the in-order successor's value. This effectively moves the problem of deletion to a node with fewer than two children.
   - Update the current node to the in-order successor and the parent to the in-order successor's parent for the next steps.
5. **Handle node with one or no child**:
   - Determine the single child of the node (if it exists) by checking if the left child is not null. Otherwise, use the right child (which could also be null if the node has no children).
   - If the node to be deleted is the root (i.e., parent is null), then the root should be updated to this single child.
   - If the node to be deleted is not the root, update the parent's link to bypass the node to be deleted, effectively removing it from the tree. This involves setting the appropriate child pointer (left or right) of the parent to the single child of the node to be deleted.
6. **Return the possibly updated root of the tree,** as the deletion of a node might have changed the tree's root (especially in cases where the root itself is deleted and has a single child or no child).

Algorithm Walkthrough

Let's consider the input:



- **Key to Delete**: `3`

Steps:

1. **Start with the root:** `root = 8`. The root is not null, so we proceed to find the node with the key to delete.

2. **Search for the node with value `3`:**
   - Start from the root. Current node (`cur`) is `8`, and its value does not match the key. Parent (`parent`) is initially `null`.
   - Move to the left child of `8` since `3` is less than `8`. Now, `cur = 3` and `parent = 8`.
   - The value of `cur` matches the key. Node to delete is found.

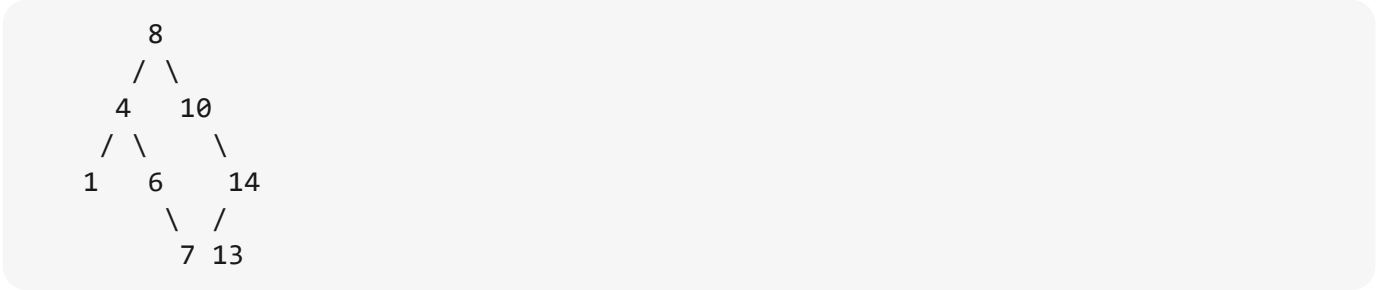
3. **Check children of node `3`:**
   - Node `3` has two children (`1` and `6`), so we need to find the in-order successor in its right subtree.

4. **Find in-order successor of node `3`:**
   - The in-order successor is the smallest node in the right subtree of `3`. We start at node `6`.
   - Move to the left child of `6` to find the smallest node, which is `4`. No further left children, so `succ = 4` and `succParent = 6`.

5. **Replace node `3` with its in-order successor `4`:**
   - Swap values: Now, instead of deleting `3`, we delete `4` after copying its value to `3`. So, the value of `cur` (which is `3`) becomes `4`.
   - Adjust pointers to delete `4`. Since `4` has no left child, we directly proceed to its deletion.

6. **Handle deletion of `4`** (which now effectively takes the place of `3` for deletion due to value swap):
   - Node `4` is a leaf (no children), so we simply remove it by updating its parent's link.
   - `4` is the left child of `6`. So, we set the left child of `succParent` (`6`) to `null`.

7. **Final Tree Structure:**



Code

Java

```java
import java.util.LinkedList;
import java.util.Queue;

// class TreeNode {
//     int val;
```

```
6   //     TreeNode left, right;
7
8   //     TreeNode(int x) {
9   //         val = x;
10  //     }
11  // }
12
13  public class Solution {
14      public TreeNode deleteNode(TreeNode root, int key) {
15          if (root == null) return null;
16
17          // Find the node to be deleted and its parent.
18          TreeNode parent = null;
19          TreeNode cur = root;
20          while (cur != null && cur.val != key) {
21              parent = cur;
22              if (key < cur.val) cur = cur.left;
23              else cur = cur.right;
24          }
```

Run Code Result | Stdout

You must run your code first

Execute

Complexity Analysis

Time Complexity

The time complexity of the algorithm is $O(h)$, where h is the height of the binary search tree (BST). In the worst-case scenario, the node to be deleted is located at the bottom of the tree, requiring a traversal from the root to a leaf node.

Space Complexity

The space complexity of the algorithm is $O(h)$ for the system's call stack due to recursion during the search for the node to be deleted, as well as finding the in-order successor if necessary.

← Previous

Delete Node In A BST (Medium)

Next →

Sum Of Subarray Minimums (Medium)

Mark as Completed